

Chapter 4 · Lesson 3 — useMemo and Derived Data

Chapter 4 · Lesson 3 — useMemo and Derived Data

*Goal: understand useMemo deeply enough that you know not just **how** to use it, but **when not to**. By the end of this lesson, you can defend every memoization decision in this codebase.*

What is useMemo?

useMemo is a React hook that **caches the result of a computation across renders**. The function inside it runs only when its dependencies change.

```
const teamsWithScores = useMemo(() => {  
  return TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) })))  
  .sort((a, b) => b.scores.total - a.scores.total);  
}, []);
```

Two arguments:

1. **A function** that produces the value.
2. **A dependency array** that tells React when to re-run.

If the dependency array is `[]` (empty), the value is computed once and reused forever. If it's `[someValue]`, React re-runs whenever `someValue` changes.

In our orchestrator, `TEAMS` is a module-level constant — it never changes during the app's lifetime. So `[]` is correct: compute the decorated team list once and reuse it.

What does “across renders” mean?

Recall from Lesson 1: every state change re-runs the component function from the top. Without `useMemo`, this:

```
const teamsWithScores = TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t)
  .sort((a, b) => b.scores.total - a.scores.total);
```

...runs on **every render**. Click a tab → re-render → recompute scores. Type in a search box → re-render → recompute scores. Resize the window → re-render → recompute scores.

For 8 teams × 31 picks each, the wasted work is microseconds. **Performance isn’t the main reason to memoize this.** The main reason is something subtler: **reference equality**.

Reference equality vs value equality

In JavaScript, object and array equality is **by reference**, not by content:

```
const a = [1, 2, 3];
const b = [1, 2, 3];
a === b;           // false! different references
a === a;           // true – same reference

const c = a;
c === a;           // true – same reference (just an alias)
```

Two arrays with the same content are NOT equal in JS. Only the literal same array (same memory address) is.

This matters because **React uses reference equality** to decide whether props or state changed. If you pass `teamsWithScores` as a prop:

```
<StandingsTab teams={teamsWithScores} />
```

...and `teamsWithScores` is a **new array on every render** (because we recomputed it), then React thinks the prop changed every render — even though the contents are identical. `<StandingsTab>` and everything below it re-renders unnecessarily.

`useMemo` with `[]` deps **freezes the reference**. The exact same array object is reused across renders. React’s reference comparison sees “same reference, no change, skip re-render of children.” (With `React.memo` / proper reconciliation, anyway. We’ll talk about

React.memo in a sec.)

Why this matters: the cascade

Without useMemo, here's what happens when the user clicks a tab:

1. Orchestrator re-renders.
2. `teamsWithScores = TEAMS.map(...)` – new array, new reference.
3. `<StandingsTab teams={teamsWithScores} />` – React sees the prop is “different”.
4. `<StandingsTab>` re-renders.
5. Inside, `<LeagueTable teams={teams} />` – new prop reference cascading down.
6. `<LeagueTable>` re-renders.
7. Each `<tr>`'s child components re-render.

Every component below the orchestrator re-renders, on every state change, even when the data is identical. **Cascading wasted work.**

With useMemo, the reference is stable. React compares old prop to new prop, sees they're the same reference, and *can* skip re-rendering. (Whether it actually skips depends on whether components are wrapped in `React.memo` – see below.)

For our app's size, the actual performance difference is invisible. But the **mental model** is what matters: **stable references mean predictable rendering**. Senior engineers care about this even when it's not measurably slow.

When NOT to memoize

This is where most beginners go wrong. They learn useMemo and start wrapping every computation in it. **Don't.**

Three situations where useMemo is **wrong**:

1. The computation is cheap.

```
// 🚫 Pointless – addition is faster than the memoization machinery
const total = useMemo(() => count + 1, [count]);
```

```
// 📌 Just compute it
const total = count + 1;
```

useMemo itself has overhead. React stores the value, the dependencies, compares dependencies on every render. For trivial computations, the memoization machinery costs **more** than the computation it's avoiding.

Rule of thumb: **don't memoize anything cheaper than calling a function**. Arithmetic, string concatenation, simple property access — leave them alone.

2. The result isn't being passed to a child or used in another hook's deps.

```
// 📌 Pointless — totalGames is only used in this render
function Component() {
  const games = useState(...)[0];
  const totalGames = useMemo(() => games.length, [games]); // why?
  return <div>{totalGames}</div>;
}

// 📌 Just compute it
function Component() {
  const games = useState(...)[0];
  return <div>{games.length}</div>;
}
```

Memoization protects against unnecessary work *somewhere else*. If the value never leaves the current render, there's nothing to protect.

3. The dependencies change every render anyway.

```
// 📌 The dep array contains a new object every render — useMemo never hits i
const config = useMemo(() => ({ apiKey, timeout: 5000 }), [{ apiKey, timeout

// 📌 same problem with inline functions
useEffect(() => fetchData(config), [config]); // re-runs every render
```

If your deps change every render, you've defeated the purpose of memoization.

The senior rule

Memoize when the result is referentially unstable AND that instability causes waste downstream. In other words: when the result is passed to a child component, used in another hook's deps, or put in Context. Otherwise leave it alone.

For our orchestrator, `teamsWithScores` is passed to **four different tab components** as a prop. That's clear downstream waste if the reference flickers. Memoize.

What about `React.memo`?

This is a separate but related tool. Different name, related job.

```
import { memo } from 'react';

const StandingsTab = memo(function StandingsTab({ teams, onTeamClick }) {
  // ...
});
```

`React.memo` wraps a component and tells React: *“only re-render this component if its props’ references actually changed.”* Without memo, every parent re-render re-renders the child even if props are the same reference.

`useMemo` keeps a value stable. `React.memo` makes a component skip re-renders. They're often used together: memoize the props, then memo the component, then the component skips re-renders when the parent re-renders for unrelated reasons.

In our codebase we don't use `React.memo` because the app is small enough that the unnecessary re-renders are imperceptible. But you'll see it in larger codebases — it's worth knowing the pattern.

Interview question: *“What’s the difference between `useMemo` and `React.memo`?”* Answer: *“`useMemo` caches a **value** across renders. `React.memo` wraps a **component** and skips re-renders when props don’t change. They’re often used together: stable values via `useMemo`, plus component skipping via `React.memo`.”*

What about `useCallback`?

`useCallback` is `useMemo` for functions. It caches a function reference across renders.

```
// Without useCallback: new function every render
<StandingsTab onClick={(t) => { setSelectedTeam(t); setActiveTab('predic

// With useCallback: stable reference
const handleClick = useCallback((t) => {
  setSelectedTeam(t);
  setActiveTab('predictions');
}, []);
<StandingsTab onClick={handleClick} />
```

Same use case as `useMemo`: protect downstream children that compare props by reference. **Same rules apply** — only worth it if the resulting function is passed somewhere or used in another hook's deps.

In our orchestrator we don't bother with `useCallback`. Why? Because `<StandingsTab>` is rendered fresh on every active-tab match anyway (it only mounts when `activeTab === 'standings'`); the inline function reference doesn't ripple beyond it. Optimizing here would be theater, not a real win.

If we ever introduced `React.memo` on `<StandingsTab>` we'd want to wrap the callback in `useCallback`. Until then, no.

When the dependency array is wrong

The dependency array is **the most common bug source** in `useMemo` and `useEffect`. Two failure modes:

Missing a dependency

```
// BUG — depends on `prefix` but doesn't list it
const greeting = useMemo(() => `${prefix} ${name}`, [name]);
```

If `prefix` changes but `name` stays the same, the memo doesn't recompute and you get a stale value. ESLint's `react-hooks/exhaustive-deps` rule catches this — **don't disable it**.

Including a dependency that breaks memoization

```
// BUG – config is recreated every render, so this never hits the cache
function Component({ config }) {
  const result = useMemo(() => expensiveCalc(config), [config]);
}
```

If the parent passes `config={{ a: 1 }}` inline, `config` is a new object every render. The memo never caches. Either memoize `config` in the parent or pass primitives instead of objects.

The right way

The eslint rule is your safety net. Fix what it tells you. **Trust ESLint over your gut** for hooks-related rules — they encode subtle React behavior that's easy to miss.

Why we use `useMemo` for `teamsWithScores`

Recap of our orchestrator:

```
const teamsWithScores = useMemo<TeamWithScores[]>(() => {
  return TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))
    .sort((a, b) => b.scores.total - a.scores.total);
}, []);
```

Justification:

- **Passed to four tabs as a prop.** Stable reference helps any future `React.memo` we add.
- **Computed from a module-level constant** (`TEAMS`). Empty dep array `[]` is correct — never recompute.
- **Marginally cheaper** — saves $8 \times 31 = 248$ microsecond-y `scorePick` calls per render.

If `TEAMS` were loaded from an API and stored in `useState`, the dep array would need to include the state variable: `useMemo(() => ..., [teamsRaw])`. Whenever the API state updated, the memo would recompute.

Why we DON'T use useMemo for the tabs array

```
const tabs: { id: TabId; label: string; icon: typeof Trophy }[] = [
  { id: 'standings', label: 'Standings', icon: Trophy },
  // ...
];
```

This array is also recreated every render. Why don't we memoize it?

Three reasons:

1. **It's not passed as a prop anywhere** — it's only consumed within the orchestrator's own JSX. No downstream cascade.
2. **It's small** — five entries, no real computation cost.
3. **Memoizing it would add boilerplate without measurable benefit.**

This is the senior call: **memoize where it matters, leave where it doesn't**. The result is more readable code and no measurable performance hit.

A common pitfall: memoizing JSX

You may see this in the wild:

```
// Anti-pattern — memoizing JSX is rarely worth it
const heroSection = useMemo(() => (
  <div>...</div>
), []);
return <div>{heroSection}</div>;
```

JSX inside useMemo is a **code smell**. JSX is cheap to evaluate. The “memoization” doesn't actually skip re-renders of the inner components; React re-renders them based on their own props. You've added complexity and gained nothing.

If you want to skip re-rendering of a component, **use React .memo on the component itself**, not useMemo on its JSX.

What about state machines and useReducer?

useReducer is useState's big sibling — useful when state has complex transitions:


```
const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: 'TEAM_SELECTED', team });
```

Pros: state transitions are explicit (each action type is documented). Cons: more boilerplate.

For our app, `useState` is the right call because we have two simple state cells. If `selectedTeam` updates ever became coupled to other state changes (e.g. “selecting a team also clears a filter and updates the URL”), `useReducer` would start to pay for itself.

Interview tip: “*useState vs useReducer?*” Answer: “*useState for independent values. useReducer when state transitions become complex enough that an explicit action type makes the code clearer. They’re both stored as state — useReducer is just a different ergonomic interface.*”

The whole orchestrator’s hook list

Re-read our orchestrator with hooks in mind. The hook calls in order are:

1. `useState<TabId>('standings')` — first state cell.
2. `useState<TeamWithScores | null>(null)` — second state cell.
3. `useMemo(() => ..., [])` — first memo cell.

Three hook calls. **They run in this exact order on every render.** That’s why “rules of hooks” exist — React indexes them by order.

If you moved any of them inside an `if` block, React would lose track of the indexing on renders where the condition was false. Hence the rule: **always at the top level of the function body.**

Vibe prompt you would have used

“Wrap the `teamsWithScores` computation in the orchestrator with `useMemo`. The dependency array is empty (`TEAMS` is module-level). Add the explicit type parameter `<TeamWithScores[]>`. Don’t memoize the `tabs` array (it’s only used inside this component). Don’t add `useCallback` for the inline event handlers (no `React.memo`’d children consume them). Add a one-line comment explaining why `useMemo` with `[]` deps.”

The big picture

After this lesson you should be able to answer these in interviews:

- *“What does `useState` do?”* — Allocates a state cell. Returns the current value and a setter. Setter triggers a re-render.
- *“What does `useMemo` do?”* — Caches a computed value across renders. Re-runs only when deps change.
- *“What does `React.memo` do?”* — Wraps a component to skip re-renders when props haven’t changed by reference.
- *“What does `useCallback` do?”* — Like `useMemo` but for functions.
- *“When should I memoize?”* — When a value is passed as a prop to many children, used in another hook’s deps, or put in Context, AND its reference would otherwise be unstable.
- *“What’s the difference between state and derived data?”* — State is the source of truth. Derived data is computed from state. Don’t store derived data in state.
- *“Where should state live?”* — At the lowest common ancestor of every component that reads or writes it.
- *“Why are hooks ordered by call sequence?”* — Because React tracks them by call index. They must be called in the same order every render.

That list is **most of the React-knowledge surface area for a mid-level interview**. If you can answer all eight crisply with a real example from this codebase, you’re hireable.

CHECK YOURSELF

1. **Why does `useMemo<TeamWithScores[]>(() => ..., [])` use an empty dependency array?** What would change if TEAMS were loaded from an API into a `useState` instead?
2. **What’s the difference between `useMemo` and `React.memo`?** Give one scenario where you’d use one but not the other.
3. **A teammate wraps every computation in `useMemo` “for performance.” What do you tell them?** Be specific about when memoization helps and when it just adds complexity.
4. **You add `console.log('memo running')` inside the `useMemo` callback. After app boots and the user clicks 4 different tabs, how many times does it log?**
5. **The `tabs` array is recreated every render. Why is this fine?** What would change if you started passing `tabs` as a prop to a memoized `<TabBar>` component?

When you've answered these, you've finished Chapter 4. Onward to **Chapter 5 – Composition** – where we break the big tabs into reusable components.